

CPSC 455 Milestone 1 – Project Design and Mockups

- Team Name: Team IDK
- Project or App Name: “SmartAPT”
- List of Team Members:
 - Janet Song (65964959)
 - James Donham (41160573)
 - Zhuoyan Qiu (39795802)
 - Gale Kanegae Penha (71067334)

Project Summary

- Project Description: Briefly describe what your project does.
 - Apartment building and facilities management, including functions for:
 - Viewing the availability of, booking and paying for shared facilities via credit system (e.g. laundry)
 - Creating and resolving maintenance requests
 - Writing and sharing notices
 - Managing shared facilities, residents and building access as a building managers
- Inspirations: List any websites or other applications that are inspiration for this project, and briefly describe or show what elements are borrowed from the inspiration.
 - Existing rental apartment services platforms (usually from corporate landlords), e.x: MyVista Tenant Portal App
 - For laundry-related functions: Coinamatic Mobile
- Target Users: Who are your target users and what will they use this app for?
 - Targets are residents and staff of apartment buildings
 - Residents will use the app as a single dashboard to book facilities, make maintenance requests, receive notices from resident managers, etc.
 - Staff will use the app to review maintenance requests, send notices to all residences, and manage facility usage through a paid credit system.
- Data: What type of data will this project collect, use, and store?
 - Precise addresses (i.e: building, suite no#)
 - Live Activity Data (for data visualizations), e.x: number of bookings made on a particular day/time.
 - Maintenance Requests/Reports
 - Reports and Notices
- Non-trivial features: What features does your team consider to be non-trivial and why.
 - Mock integration with IoT APIs, which will involve mocking hardware for devices such as laundry machines.
 - Multiple authenticated views, as residents and building managers should be able to view different information in the system based on their accounts. These views should be made such that no unnecessary personal information can be obtained

- from a non-authenticated view or from an unrelated account (e.g. a resident should not be able to see the payment information of another resident).
 - Submission, ranking, and handling of maintenance requests and notices
- Standard features: What are the standard features of your project (e.g., user authentication)?
 - User authentication
 - Laundry credits -> When a user uses a laundry service -> it uses up a credit.
 - Forms to enter personal information and payment information
 - Resident and building manager views for shared facilities.
 - Resident and building manager views for maintenance requests.
 - Resident and building manager views for notices.
 - Payment portal for residents for shared facilities.
 - Building manager view for resident management.
 - Database storage and organization of apartment related information for both types of views (resident, building manager)
- Risks - data (e.g., product data, reviews, movies), privacy (e.g., storing PII, personal info, working with minors)
 - Privacy: Storing personal information such as precise addresses.
 - Data: Managing multiple authenticated views securely and privately.

Feedback

Lack of clarity on data visualization

- **Source:** Peers, TAs.
- **Description:** In the project pitch presentation and the M0 submission, the use of charts for visualization of usage trends was mentioned without providing specific details on how they will be implemented or how the underlying data will be processed. No details have been given on the collection of live activity data.
- **Action:** We specify more precisely the scope, implementation and employed data for the data visualization charts as follows:
 - **Scope:** The charts are to be visible for both building managers and residents in the “shared facilities” pages, where there is one chart per automatically tracked facility (i.e. facilities which have been set up with IoT integration with our system) indicating an estimate of how busy it is (meaning, for example, that a shared laundry room with multiple machines has only one chart that shows occupancy based on how many machines are in use).
 - **Implementation:** Data will be maintained in a MongoDB database, as with the other data collected and stored by our system, and be collected through the IoT APIs, which should be able to indicate whether a certain device or facility is in use by one or more people at any given time. On the user interface, charts will be drawn using the D3.js library, and each chart will correspond to a facility, which may contain one or more devices.

- **Employed data:** Information from IoT devices indicating whether a given device or facility is in use and by how many people will be collected and timestamped as the device or facility is used. A rolling average within a certain period of time (e.g. 1 month) will then be taken from this data grouped by time of day and day of the week, and this average will be represented in a column chart form. As it is an average and the data will be presented in an anonymous manner, this is not considered critical in terms of data security and privacy.

Notification System for Bookings and Updates

- **Source:** Peers
- **Description:** Multiple reviewers independently suggest adding an alert or notification system to the platform. More specific will be (email or push notifications) for maintenance updates, payment alerts, and booking reminders. One reviewer further pointed out that building notices are easy to miss when sent by email, and another suggested SMS alerts notifying users a few minutes before their booked time slot begins or ends to prevent accidental no-shows or overruns on shared facilities.
- **Action:** We have decided to add email notifications as a stretch goal feature(TBD). Specifically, automated email notifications will be triggered for the following events, and implemented as described below:
 - **Scope:** Email notifications will be available to all user roles. Residents will receive notifications for booking confirmations, upcoming slot reminders (e.g., 10 minutes before a booked facility slot begins), maintenance request status changes, and new building-wide notices. Building managers will receive notifications when a new maintenance request is submitted or when a resident completes a payment.
 - **Implementation:** Notifications will be triggered by backend events (e.g., a booking being created, a maintenance request status being updated) and sent via a third-party email service such as Nodemailer or SendGrid. The backend will expose a notification service module that listens for these events and dispatches the appropriate email template. No additional frontend work is required beyond what is already planned, as notifications are sent passively in the background.
 - **Employed data:** Each notification email will draw from existing data already stored in MongoDB, such as the resident's name and email, the booked facility name and time slot, or the maintenance request description and updated status. No additional data collection is introduced solely for the purpose of notifications, and no sensitive financial data will be included in notification content.

Explicit plan on integrating with Payment API Stripe:

- **Source:** Peers, TAs
- **Description:** We have been asked to give more explicit detail on testing transactions and securing transactions with Stripe API.
- **Action/Decisions:**

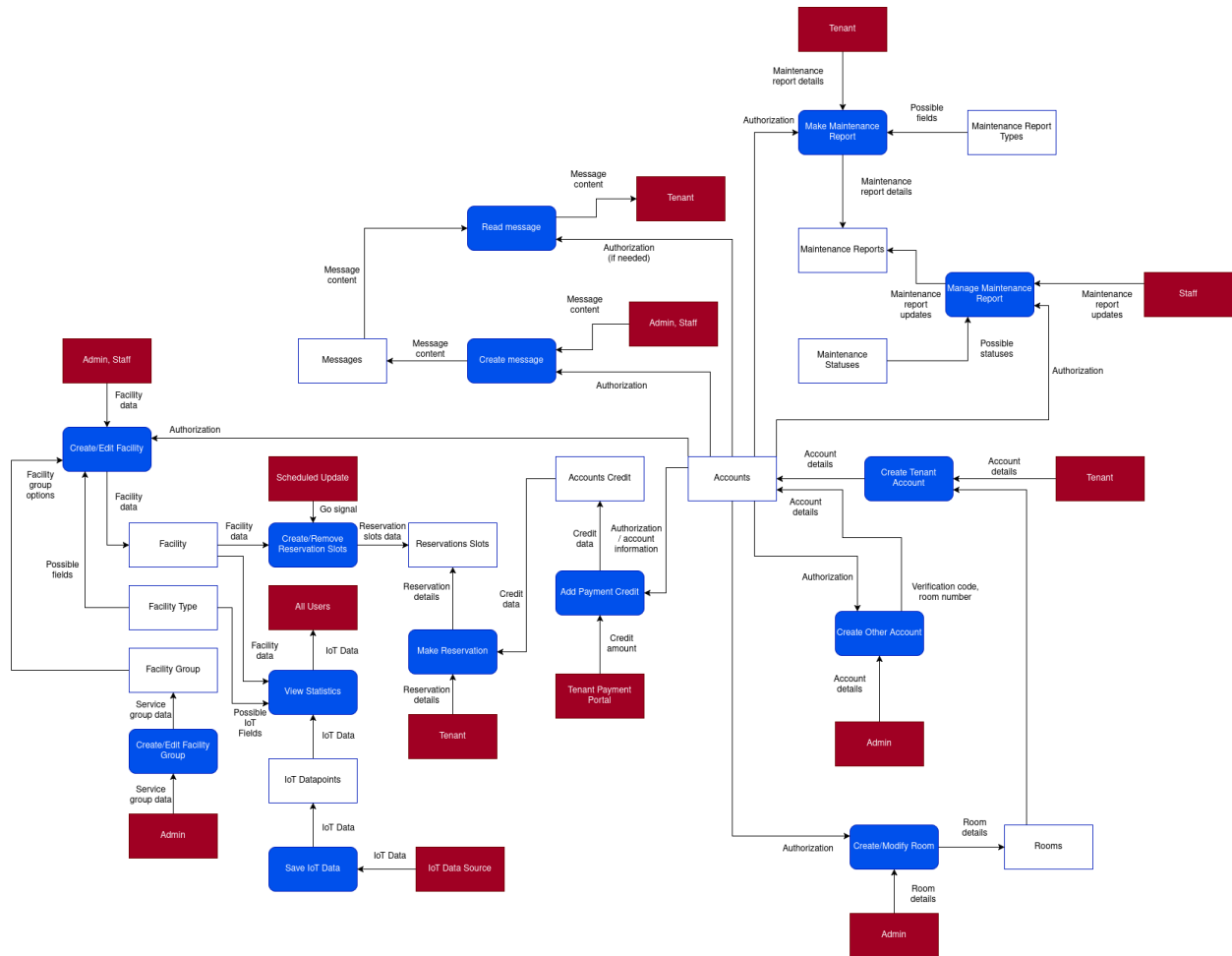
- We have decided to remove the option of integrating with Stripe API upon advice from TAs as historically in the past previous projects that tried to integrate with payment APIs had to pay to use their services.
- Stripe API does provide free sandbox testing however, and all payment information should never reach our backend nor be collected directly through our frontend as Stripe handles all security.
- Still in terms of project scope, we believe implementing multiple views and integrating IoT APIs and the Payment API might be too much on our plate.
- After further discussion with the TA the team has decided to remove payment API integration from our list of priorities and focus on IoT API integration and multiple views.

Explicit plan on integrating with IoT APIs and mocking IoT API calls:

- **Source:** Peers, TAs
- **Description:** We have been asked to give more explicit detail on what IoT API we are using, how to integrate IoT APIs in our backend, test and mock IoT API calls.
- **Action/Decisions:**
 - Our project lead has decided on using Eclipse Mosquitto which is an open source MQTT broker that can make connections with multiple IoT devices. To configure this broker and send and receive messages to and from it, we will use mqtt.js, which is a library built for specifically interacting with this broker.
 - Description on how the communication with this broker is implemented is discussed in detail in the architecture diagram in M1.
 - A team member will create a CLI program to represent an IoT device that will be connected to the broker. Our backend will also be connected to this broker and make requests to the program through this broker.
 - To test requests made to the broker, we will use Jest and Testcontainers. Test containers is a free open-source testing framework that allows us to set up the test environment with all necessary dependencies (so long they can run in a docker container) so we will most likely be using this to test communication between the backend and the broker.

Dataflow Diagram

Below is a dataflow diagram that shows major paths of dataflow.



Legend

- Red: External actor or API
- Blue Process
- Outline: Database store

REST API

Our application will use a REST API as specified below. Brackets are used to indicate placeholders.

- /auth/login
 - POST: Attempt to log into an account and get a new JWT.
- /auth/logout
 - POST: Removes logged in status from current JWT.
- /auth/verify
 - GET: Attempt to verify logged in status using JWT.
- /accounts
 - POST: Creates a new account
- /accounts/{accountID}
 - GET: Gets details of an account.
 - DELETE: Removes an account.
 - PUT: Edits details of an account.
- /account-credit/
 - POST: Create a new account credit wallet
- /account-credit/{accountID}
 - GET: Gets the content of the wallet associated with the account.
 - DELETE: Removes wallet associated with the account.
- /accounts/{accountID}/add-value
 - POST: Add a certain amount to the wallet associated with the account.
- /IoT-Data/
 - GET: Gets IoT data based on request.
 - POST: Adds an IoT datapoint.
- /residents/
 - GET: Gets all residents.
 - POST: Creates a new resident.
- /residents/{residentID}
 - GET: Gets profile for the provided resident.
 - DELETE: Removes the provided resident.
 - PUT: Edits the provided resident's profile.
- /maintenance-requests/
 - GET: Gets all maintenance requests.
 - POST: Creates a new maintenance request.
- /maintenance-requests/{maintenanceRequestID}
 - GET: Gets details for the provided maintenance request.
 - DELETE: Removes the provided maintenance request.
 - PUT: Edits or resolves the provided maintenance request.

- /notices/
 - GET: Gets all notices, and according to the ID of the current household, dynamically return whether the announcement is read or unread to the user.
 - POST: Creates a new notice.
- /notices/{noticeID}
 - GET: Gets details for the provided notice.
 - DELETE: Removes the provided notice.
 - PUT: Edits or marks the provided notice as read.
- /rooms/
 - GET: Gets all rooms.
 - POST: Creates a new room.
- /rooms/{roomID}
 - GET: Gets details for the provided room.
 - DELETE: Removes the provided room.
 - PUT: Edits or reset the verification code for the provided room.
- /facilities/
 - GET: Gets all facilities.
 - POST: Create a new facility.
- /facilities/{facilityID}
 - GET: Gets details for the provided facility.
 - DELETE: Removes the provided facility.
 - PUT: Edits the provided facility.
- /facility-types/
 - GET: Gets all facility types.
- /facility-types/{facilitygroupID}
 - GET: Gets details for the provided facility type.
- /facility-groups/
 - GET: Gets all facility groups.
- /facility-groups/{facilitygroupID}
 - GET: Gets details for the provided facility group.
- /reservations/
 - POST: Creates a new empty reservation slot.
 - GET: Gets all facility reservations.
- /reservations/{reservationID}
 - GET: Gets details of a specific reservation.
 - DELETE: Removes a specific reservation.
 - PUT: Edits details of a specific reservation.
- /dashboard/
 - GET: Get the dashboard overview data of the currently logged-in household

Backend

The data for the app will be stored in a MongoDB instance. This will contain all of the data needed for maintenance requests, IoT functionality and multi-view functionality. If we add custom profile pictures these will be stored in a folder on the backend Node.js app.

The following is a list of database entities we plan on using, along with the data we plan on storing for each entity in the MongoDB instance. It also shows if and which API routes will be able to access the database entity and what data the API route will return.

- Account
 - Fields
 - ID/Account number (String)
 - Name (String)
 - Email (String)
 - Password hash (String)
 - Roles (String array)
 - API Routes
 - /auth/login
 - POST: Access username and password.
 - /auth/verify
 - GET: Check JWT against username, return roles.
 - /accounts
 - POST: Creates a new account with Name, Email, Password, Roles.
 - /accounts/{accountID}
 - GET: Gets ID, Name, Email, Roles of an account.
 - PUT: Edits Name, Email, Roles of an account.
 - /residents/
 - GET: Gets all accounts with tenant role.
 - /residents/{residentID}
 - GET: Gets profile for the provided resident.
 - DELETE: Removes the provided resident.
 - PUT: Edits the provided resident's profile.
- Account Credit
 - Fields
 - ID (String)
 - Linked account ID (String)
 - Credit (Integer)
 - API Routes
 - /account-credit/
 - POST: Create a new account with Linked account ID.
 - /account-credit/{accountID}
 - GET: Gets the value of the Credit of account wallet.

- /account-credit/{accountID}/add-credit
 - POST: Add a certain amount to the Credit of the wallet associated with the account.
- IoT Datapoint
 - Fields
 - ID (String)
 - IoT device ID (String)
 - Time recorded (Date time)
 - Dynamic value fields...
 - API Routes
 - /IoT-Data/
 - GET: Gets IoT Dynamic value fields, Time recorded, ID, IoT device ID based on request.
 - POST: Adds an IoT datapoint with IoT device ID and Dynamic value fields.
- Room
 - Fields
 - ID (String)
 - Name (String)
 - Verification code (String)
 - API Routes
 - /rooms/
 - GET: Gets ID of all rooms.
 - POST: Creates a new room based on Name.
 - /rooms/{roomID}
 - GET: Gets ID, Name, Verification code for the provided room.
 - PUT: Edits the Name or reset the Verification code for the provided room.
- Message
 - Fields
 - ID (String)
 - Title (String)
 - Content (String)
 - Read (Boolean)
 - Author account ID (string)
 - API Routes:
 - /notices/
 - GET: Gets all ID, Title, Content, Read, Author account ID, and according to the ID of the current household, dynamically return whether the announcement is read or unread to the user.
 - POST: Creates a new notice with Title Content, Author account ID.

- /notices/{noticeID}
 - GET: Gets ID, Title, Content, Read, Author account ID for the provided notice.
 - PUT: Edits the Title, Content, Read of the provided notice as read.
- Facility
 - Fields
 - ID (String)
 - Name (String)
 - Description (String)
 - Type (String)
 - Facility group IDs (String array)
 - Reservation start time (Number)
 - Reservation end time (Number, optional)
 - Reservation length (Number)
 - Linked IoT device ID (String, optional)
 - Dynamic fields...
 -
- Facility Type
 - Fields
 - ID (String)
 - Is IoT available (Boolean)
 - Field name and type list (String tuple array)
 - IoT Data name and type list (String tuple array, optional)
- Facility Group
 - Fields
 - ID (String)
 - Name (String)
- Reservation Slot
 - Fields
 - ID (String)
 - Linked facility ID (String)
 - Start time (Date time)
 - Duration (Number)
 - Booked (Boolean)
 - Booked account ID (String)
- Maintenance Report
 - Fields
 - ID (String)
 - Name (String)
 - Description (String)
 - Type (String)
 - Status (String)

- Completed (Boolean)
 - Dynamic fields...
- Maintenance Report Type
 - Fields
 - ID (String)
 - Name (String)
 - Field name and type list (String tuple array)
- Maintenance Report Status
 - Fields
 - ID (String)
 - Name (String)
 - Sort order (Number)

We also plan on using the MQTT IoT protocol. This is a machine to machine network protocol that is commonly used with IoT devices. This protocol consists of a server known as an MQTT broker, along with many client pieces of software. These clients can either be devices, or software like our backend. Clients can publish and subscribe to topics managed by the MQTT broker. The broker distributes the data published by clients to the clients that are subscribed.

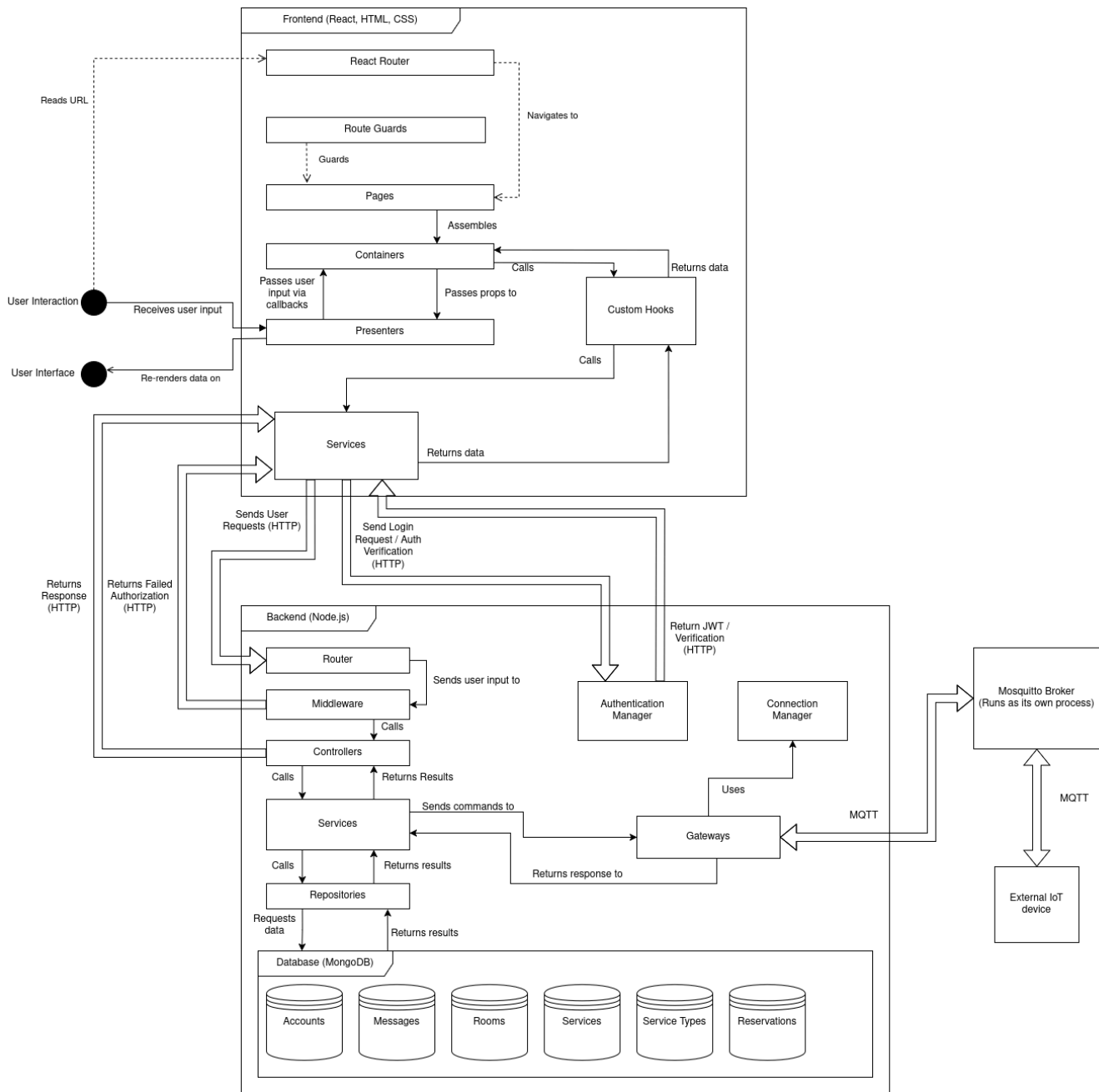
For our project we plan on using the MQTT broker known as Eclipse Mosquitto. This is an open source, free MQTT broker, available as a docker image on docker desktop. For interacting with the broker, we will use the 'mqtt' npm library for Node.js. This will be used both for the main app backend server, as well as the IoT device mocks.

The data we plan to receive through the MQTT IoT is the following:

- Washing machine
 - Machine status, on, off
 - Remaining time on cycle
- Dryer
 - Machine status, on, off
 - Remaining time on cycle

Project Architecture:

Architectural Diagram:



Below is a brief description of the components in the diagram and a list of their functions.

Frontend:

Since we have decided to build the frontend using React, the Container-Presenter pattern will be used since that is a pattern that is designed with React in mind.

List of components in Frontend:

- Route Guards (token check, role check, redirect logic, runs before render):
 - One-time checkpoint.
 - Rendered automatically and triggered by navigating to a page.
 - Checks if the JWT (JSON Web Token) is stored in local storage.
 - Verifies the role of the logged in user by reading JWT from local storage.
 - Only reads from local storage or App-wide state, no API calls made.
 - 2 main guards (runs before page renders):
 - Checks if anyone is logged in (Authentication Guard).
 - Checks if the logged in user has the right role (Role Guard).
- React Router
 - Where all routes are defined in a central place.
 - Defines all routes and their guards.
 - Routes are wrapped inside guard components
 - Types of routes: Public (no guard), Auth Guard (must be logged in), Role Guard (must be admin)
- Pages (UI Layout, assembles containers):
 - Define the overall layout of the screen
 - Decide which Container components to place.
 - Handle route-level concerns.
- Container Components (logic, role decisions, state, data fetching):
 - Runs after page renders.
 - Manages what the UI does with data.
 - Manages and access state (i.e: who is the current user, what actions should be available, how does local state change in response to user actions?) directly from the app-wide state.
 - Passes arguments to the Presenter components.
 - Knows user role.
 - Fetches data through calls to Services.
- Presenter Components:
 - Pure UI building blocks.
 - Does not know anything about state (i.e: who is logged in), data, nor routes.
 - Receives props only.
 - Only renders UI.
 - Fires events upwards when the user interacts with the interface.
 - i.e: Receives user input directly and passes straight to the Container components via props.
- Services (API Calls, Data fetching):
 - Communicates with the backend through HTTP requests
 - Send the HTTP requests: GET, POST, PUT, DELETE.
 - Return the raw results back to the Container Components.
 - Attaches JWT (JSON Web Token) to the request header.

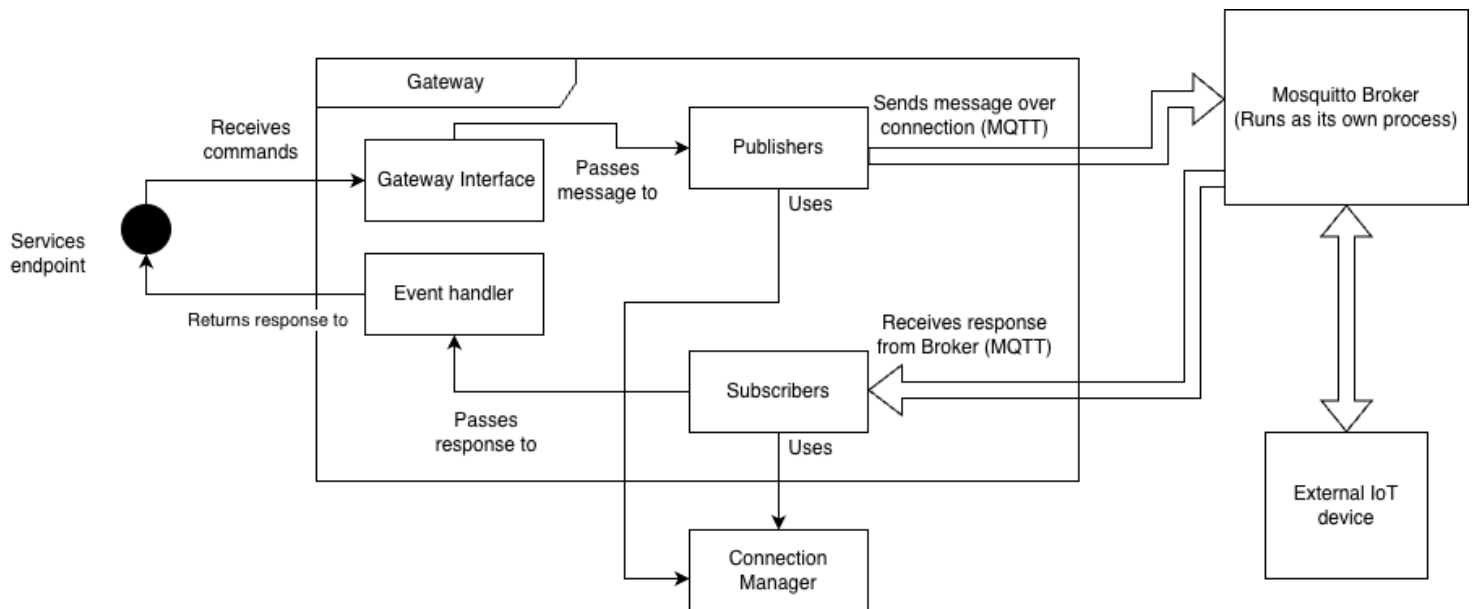
Backend:

This is also where the JWT is created and sent back from.

List of Components in Backend:

- Communicating with UI (Reads and verifies JWT on each request):
 - Router:
 - Sends incoming user requests to the Middleware.
 - Middleware (Authorization, roles, logging, errors):
 - Imports environment settings (i.e: JWT keys) from Config to verify users.
 - Intercepts requests before reaching controllers and either passes/rejects it.
 - JWT and JWT signature verification.
 - Passes user request to appropriate controller, else returns message indicating “Failed authorization”
 - Authentication Manager:
 - Intercepts user’s login request
 - Verifies the user information (id and password).
 - Generates a JWT and sends it to the client if input is valid.
 - Returns message indicating “Failed authentication” if invalid.
 - Controllers (Receive requests, send response):
 - Receives request, calls right service from Services.
 - Send back a response.
- Communicating with Database:
 - Repositories:
 - Collections of database operations (i.e: database queries)
 - Runs database queries and returns back results.
- Services (Business logic):
 - Access data from database via repositories
 - Decides when to call repositories or gateways
 - Determines sequence of operations.
- Database:
 - Stores user, admin data
 - Stores facility data, resident data, notices, messages, booking slots, etc.
- Communicating with external IoT devices:
 - Gateways (external devices):
 - Handles broker connection, message formatting, protocol translation, device commands,
 - (If using MQTT) Handles topic or channel management.
 - Receives responses from broker.
 - Broker:
 - Forwards request to physical machine
 - Returns back confirmation, status updates, etc.

Gateway Architecture:



List of Components in Gateway:

- **Gateway Interface:**
 - Receives incoming requests from Services.
 - The only endpoint exposed to other components in backend
- **Publishers:**
 - Receives request from interface.
 - Packages request into proper format to send over MQTT connection.
 - Sends messages to the broker using connection from Connection Manager.
- **Subscribers:**
 - Listens for responses from the broker.
 - Packages raw response into readable format back to Event handler.
- **Connection Manager**
 - Manages and maintains connection between backend and the broker.
 - Establishes the MQTT connection with the broker.
 - Notifies other components that connection has been established.
- **Mosquitto Broker:**
 - Runs as its own process.
 - Routes and sends messages to external IoT devices.
 - Returns response back to Subscribers.
- **Event Handler:**
 - Receives incoming broker messages from Subscribers.
 - Parses broker response and calls the appropriate handler for it.

Implementing RBAC in our project:

Since we are building an interface with multiple views and user roles, we will implement our design with the RBAC (Role-based Access Control) model in mind.

RBAC is a security model that restricts access to resources to authorized users based upon their role in the system. Our project will have strict definitions on what roles there are (Residents, Admins) and which set of permissions are associated with which role (Residents can only view their own data, Admins can view all residents data). We plan to implement authentication and authorization of user roles through JWTs (JSON Web Tokens).

Implementing User Authentication and User role authorization:

To implement user authentication, we plan to utilize JWTs (JSON Web Tokens) and send it with every future request to ensure access to only permitted resources. We will initially store the JWT key in local storage, but later use HTTP-only cookies instead. Using HTTP-only cookies has risks as they are vulnerable to XSS attacks. A better solution would be to use HTTP-only cookies paired with refresh tokens. However, this introduces an additional layer of complexity to the architecture that may be out of the project's scope. Since this is a school project, we will use HTTP-only cookies for now.

Brief Overview of User Authentication and Authorization:

These are the steps of user authentication:

1. Client makes a log-in request.
2. The app sends a HTTP log-in request to the Authentication manager in the backend.
3. After verifying the log-in information, the Authentication manager generates a JWT and sends it back to the app.
4. The app stores the JWT in client-side local storage.
5. All further client requests to the backend are sent with a JWT in its payload to the Router.
6. Middleware verifies JWT and JWT signature before granting access to protected resources.

Tech Stack:

Frontend

Languages/Libraries/Frameworks	Usage
Typescript	• Our main project language for the frontend.

	• All user interface logic will be written in TypeScript.
React.js	Implementing UI components: page routers, route guards, pages, and controllers. Examples of what would be implemented in React: • Resident dashboard. ◦ Resident view. ◦ Admin view. • Booking facilities page. • Sign-up page. • Facilities page. • And etc.
React-Redux	• Use for Application state-management. • Ensures web application will respond to changes in UI state or to user interaction.
HTML	Define content and page structure of web documents to be displayed on the browser.
CSS	Page Stylization, Responsive design, Fonts, etc.
Selenium	Web Browser Integration Testing
D3.js	Data visualizations for booking facilities page. Examples of what to visualize include: • Available slots per weekday. • Busyness of facilities per week/time

Backend

Languages/Libraries/Frameworks	Usage
Typescript	• Our main project language for the backend.

	• All business logic will be written in TypeScript.
Node.js	Running backend logic on server side. Examples of backend logic include: • Adding user accounts to the database. • Adding booked slots to the database. • Updating booked slots. • Checking status of booked facilities (e.x: checking the status of a washing machine and if it is finished) • Adding booked requests to a queue. • Checking account balance. • Checking the number of credits left. • And etc.
Express.js	• Building and defining REST API between Frontend and Backend
MQTT.js	• For IoT app integration. • Configuring the Mosquitto broker. • Setting up MQTT connection with Mosquitto broker • Sending messages to the Mosquitto broker and receiving responses.
Jsonwebtoken (JWT)	• Creating JWTs for user authentication. • Verifying JWTs.
Mocha	Creating blackbox tests for business logic alongside with Chai. Examples of business logic to test for includes: • Adding user accounts to the database. • Adding booked slots to the database.

	• Updating booked slots in the database. • Removing bookings from the database. • And etc.
Jest	• Creating integration tests within the backend. • Combined with Testcontainers to test MQTT requests made to the broker. • Test database operations. • Test data retrieval.
Chai	Creating blackbox tests for backend logic. Used primarily with Mocha.
Postman	API testing between the Frontend and the Backend
Testcontainers	• Creating integration tests. • Testing MQTT requests with the broker combined with Jest.

Database

Languages/Libraries/Frameworks	Usage
MongoDB	• Implement database that stores datasets: ◦ Messages ◦ Accounts ◦ Services ◦ ServiceTypes ◦ Rooms ◦ and etc.

UI Mockups

1. Dashboard: A central page showing building-wide announcements, current facility statuses, and shortcuts to help residents or building managers coordinate daily apartment activities

Building Portal

Dashboard

Facilities
Book

Maintenance

Notices

Credits

Settings

Help

Logout

Dashboard

Rent due
\$1,450
Due Jul 1

Open tickets
2
1 in progress

Next booking
Jun 5
Laundry 10:00

Notices
3
2 unread

Recent activity

Laundry booked — Washer 2, Jun 5 10:00
2 days ago

Maintenance ticket #12 — In progress
3 days ago

Notice: Building water shutoff Jul 3
4 days ago

Rent payment confirmed — \$1,450
Jun 1

Upcoming bookings

Laundry — Washer 2
Jun 5 • 10:00–11:00 • \$3.00

Gym
Jul 8 • 08:00–09:00 • Free

Quick actions

Book a facility

New maintenance ticket

Pay rent

Book a facility

Facility
Laundry room

Date
2026-06-05

Time
10:00

Fee: \$3.00 • charged on confirm

Cancel Confirm

New maintenance ticket

Category
Plumbing

Location
Unit / common area

Priority
Normal

Description
Describe the issue...

Cancel Submit

Pay rent

Amount
\$ 1,450.00

Card no.
**** * 4242

Expiry
MM/YY CV ***

Powered by Stripe

Cancel Pay now

[Dashboard - Resident View]

2. Help: Documentation page that offers troubleshooting guides, platform FAQs, and contact paths for resolving system errors.

Building Portal

Dashboard

Facilities
Book

Maintenance

Notices

Credits

Settings

Help

Logout

Help & Support

Search help articles...

Frequently asked questions

How do I book a laundry machine?
Go to Facilities > Book facilities, select a machine and time slot.

How do I submit a maintenance request?
Go to Maintenance, click "New request" and fill in the form.

How do I pay rent online?
Go to Payments and click "Make payment". Stripe processes your card securely.

Where can I see building notices?
Go to Notices. Unread notices are marked with a badge.

Contact support

Building manager

manager@buildingportal.com

+1 (604) 555-0100 • Mon–Fri, 9:00–17:00

How to book a facility?

How to book a facility

Step 1

Go to Facilities in the sidebar.

Step 2

Click "Book facilities".

Step 3

Select a machine that shows Available.

Step 4

Pick a date, time and click Reserve.

Step 5

Pay and confirm your booking.

Close

No results found

?

We couldn't find an article matching your search.

Please contact your building manager directly for help.

manager@buildingportal.com
+1 (604) 555-0100 • Mon–Fri 9–17

Close

[Help & Support - Resident View]

3. Settings: An account customization page where residents and managers can manage profile details, change passwords, and toggle notification channels.

The screenshot displays the 'Building Portal' interface. On the left is a sidebar menu with links: Dashboard, Facilities (with a sub-link 'Book'), Maintenance, Notices, Credits, Settings (highlighted), Help, and Logout. The main content area is titled 'Settings' and contains three sections: 'Profile' with input fields for Name (filled with 'Leo'), Email (filled with 'Q@gmail.com'), and Phone (filled with '.....'); 'Notifications' with toggle switches for Email notifications (On), Maintenance updates (On), and Notice alerts (Off); and 'Security' with a 'Change password' link and a 'Two-factor auth' toggle (Off). A 'Save changes' button is at the bottom right. To the right of the main interface are two modal windows. The first is titled 'Change password' and contains three password input fields (Current, New, Confirm) and 'Cancel' and 'Update' buttons. The second is titled 'Settings saved' and features a checkmark icon, the text 'Changes applied! Your settings have been saved.', and an 'OK' button. A footer note at the bottom left reads '[Settings - Resident View]'.

4. Logout: A simple confirmation prompt or redirect screen that securely clears the user's active session, removing their authentication token from the local environment. After finishing apartment tasks, user clicks the exit button, which safely wipes his JWT session from local storage to protect the personal data from shared devices

The screenshot shows the 'Building Portal' interface with the 'Dashboard' page selected in the sidebar menu. A confirmation modal is centered on the screen, titled 'Sign out?'. The modal text reads: 'You will be signed out of Building Portal. Unsaved changes will be lost.' Below the text are two buttons: 'Cancel' and 'Sign out'. The sidebar menu on the left includes links for Dashboard, Facilities (with a sub-link 'Book'), Maintenance, Notices, Credits, Settings, Help, and Logout. A footer note at the bottom left reads '[Logout - Confirmation Modal]'.

5. Sign up page: An onboarding portal that allows new residents to input their user profiles, create secure credentials, and link accounts to their validated apartment units.

SmartAPT

User Icon

Sign up

Name*

Access code*

E-mail address*

Password*

Re-type password*

☐ I agree with the SmartAPT terms of service.*

☐ I agree to receive e-mails about my facility reservations and maintenance requests, and building notices.

Cancel

Sign up

6. Facilities page, resident view: A booking interface for residents featuring D3.js usage charts, real-time device availability, and simple buttons to instantly reserve facility slots.

Building Portal

User Icon

Dashboard

Facilities

Book facilities

Maintenance

Notices

Credits

Settings

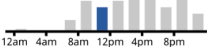
Help

Logout

Book facilities

Laundry room

Availability
Less busy than usual

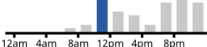


Available
(2 washers, 1 dryer)

Reserve

Event space

Availability
Busier than usual



Reserved until 3 PM

Unavailable

Swimming pool

Availability

Closed
Under maintenance

Book laundry room

Device

Washer 1

Setting

Regular cycle, hot water

Date

2026-06-01

Time

11:00

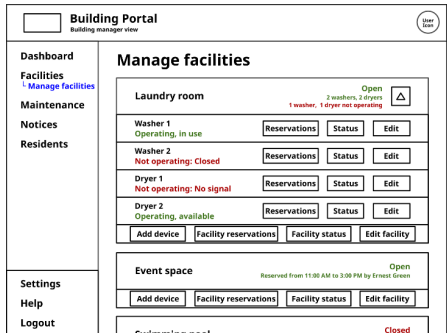
Cost

\$3.00

Cancel

Pay and reserve

7. Facilities page, building manager view: An administrative console that gives building managers full control to add, edit, or remove shared facilities and track connected IoT hardware. Property manager logs into add a newly integrated IoT dryer to the building's list, toggling the available checkbox to make it visible to the community.



Building Portal
Building manager view

Manage facilities

Laundry room Open

Washer 1: Operating, in use Reservations Status Edit

Washer 2: Not operating: Closed Reservations Status Edit

Dryer 1: Not operating: No signal Reservations Status Edit

Dryer 2: Operating, available Reservations Status Edit

Add device Facility reservations Facility status Edit facility

Event space Open

Add device Facility reservations Facility status Edit facility

Reservations for Washer 1

Monday	Tuesday
1 AM	
2 AM	
3 AM	Facility closed
4 AM	Facility closed
5 AM	
6 AM	
7 AM	San Uweidyn
8 AM	
9 AM	Nora Jimenez
10 AM	
11 AM	
12 PM	
1 PM	

Close Add reservation

Status for Washer 2

Device status: Not operating, connected
Last updated: 2026-06-01 11:05 AM by Manager

Status source: Manual

Note: Closed for maintenance

Close Save changes

Edit Washer 2

Name: Washer 2

IP address: 192.168.3.24

Device type: Laundry washing machine

Close Delete device Save changes

Edit Laundry room

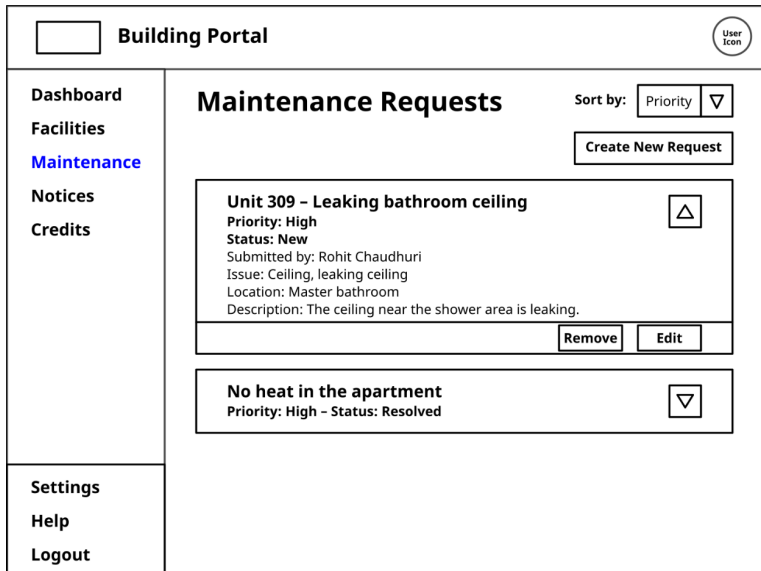
Name: Laundry room

Opening hours: Every day, 6:00 AM - 12:00 AM

Close Save changes

*The "Add Device" dialog will be identical to the above except for the title.

8. Maintenance requests page, resident view: Logging portal where residents can fill out categories and descriptions to file new work orders and monitor the live progress of past requests



Building Portal
User Score

Maintenance Requests Sort by: Priority

Create New Request

Unit 309 - Leaking bathroom ceiling △

Priority: High
Status: New
Submitted by: Rohit Chaudhuri
Issue: Ceiling, leaking ceiling
Location: Master bathroom
Description: The ceiling near the shower area is leaking.

Remove Edit

No heat in the apartment ▽

Priority: High - Status: Resolved

Create New Request

Priority: Medium

Issue type: Exhaust fan

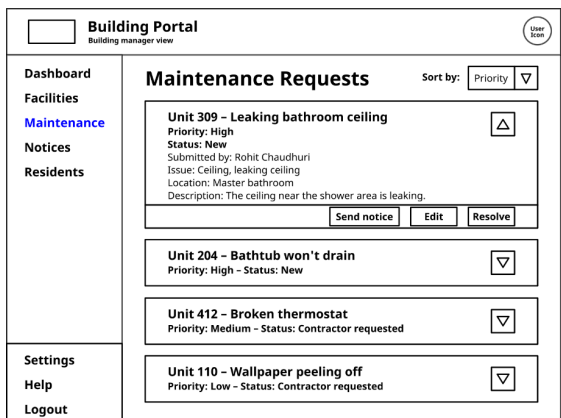
Issue: Exhaust fan broken

Location: Master bathroom

Description: The exhaust fan does not spin.

Cancel Delete Save changes

9. Maintenance requests page, building manager view: Shown a master queue for building staff to view submitted issues organized by unit number, assign tasks, and alter completion statuses



Building Portal
Building manager view

Maintenance Requests Sort by: Priority

Unit 309 - Leaking bathroom ceiling △

Priority: High
Status: New
Submitted by: Rohit Chaudhuri
Issue: Ceiling, leaking ceiling
Location: Master bathroom
Description: The ceiling near the shower area is leaking.

Send notice Edit Resolve

Unit 204 - Bathtub won't drain ▽

Priority: High - Status: New

Unit 412 - Broken thermostat ▽

Priority: Medium - Status: Contractor requested

Unit 110 - Wallpaper peeling off ▽

Priority: Low - Status: Contractor requested

Resolve "Leaking bathroom ceiling"

Comment:

Cancel Resolve and close

Edit "Leaking bathroom ceiling"

Priority: High

Issue type: Ceiling

Issue: Leaking ceiling

Location: Master bathroom

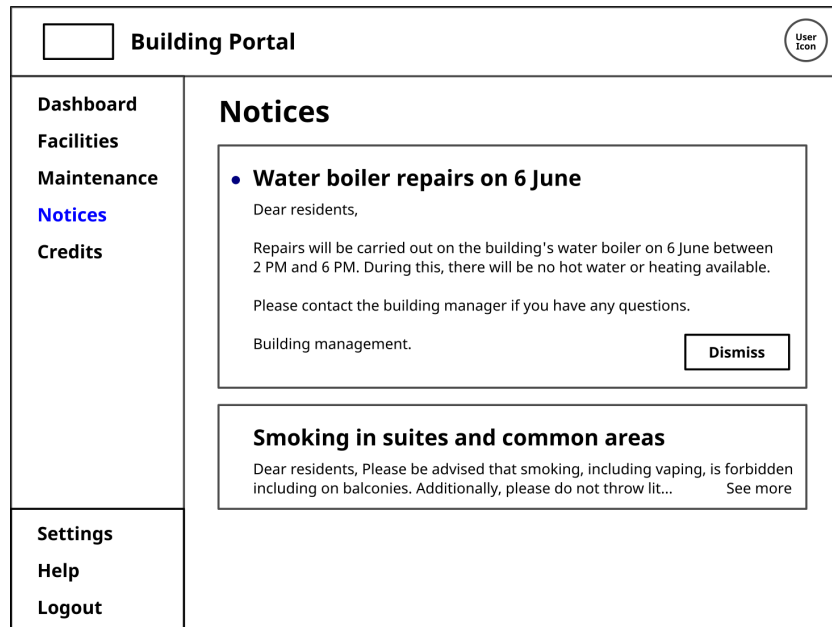
Description: The ceiling near the shower is leaking.

Status: New

Cancel Delete Save changes

The "Send notice" feature opens a dialogue identical to the "Create Notice" dialogue used for the Notices page, with the "Visibility" dropdown menu removed.

10. Notices page, resident view: A virtual bulletin board highlighting building-wide updates where unread announcements are explicitly marked to avoid missing important changes.



11. Notices page, building manager view: A publishing workspace enabling managers to broadcast new rules, schedule announcements, and purge old notices from the building database.

Building Portal
Building manager view

User Icon

Dashboard
Facilities
Maintenance
Notices
Residents

Settings
Help
Logout

Notices

Create Notice

Water boiler repairs on 6 June

Dear residents,
Repairs will be carried out on the building's water boiler on 6 June between 2 PM and 6 PM. During this, there will be no hot water or heating available.
Please contact the building manager if you have any questions.
Building management.

CancelDeleteSave changes

Smoking in suites and common areas

Dear residents, Please be advised that smoking, including vaping, is forbidden including on balconies. Additionally, please do not throw lit... See more

Create Notice

Title

Body

VisibilityAll residents

CancelCreate Notice

12. Unit management page: A manager-only directory mapping each apartment number to its current occupants and associated verification permissions.

Building Portal
Building manager view

User Icon

Dashboard
Facilities
Maintenance
Notices
Residents
Manage units

Settings
Help
Logout

Manage Units

Create Notice

Unit 101

Ian LlewelynReservationsMaintenanceAccessEdit

New resident accessEdit unit

Unit 102

Alberto JimenezReservationsMaintenanceAccessEdit

Nora JimenezReservationsMaintenanceAccessEdit

New resident accessEdit unit

Unit 103

Sally MatsumotoReservationsMaintenanceAccessEdit

New resident accessEdit unit

Access for Unit 101

Access Code:ABC123

Not yet used.

CancelGenerate access code

The "New resident access" dialogue is identical to the above, except without a resident name.

Edit: Ian Llewelyn

Name:Ian Llewelyn

Address:1234 Fir Street, Vancouver, BC Canada V2L 6G3

Unit #:101

E-mail:ianl@example.com

CancelDeleteSave changes

Edit: Unit 102

Name:Unit 102

CancelDeleteSave changes

The "Reservations" dialogue is equal to the reservations dialogue in the building manager's "Manage Facilities" page.
The "Maintenance" dialogue is equal to the maintenance dialogue in the building manager's "Maintenance Requests" page.

13. Credit page: Credit page displaying previous transaction histories alongside financial input fields that allow residents to securely add funds to their account credit wallets and also choose different methods they want.

Building Portal User Icon

Credits Sort by: Priority

Balance
\$ 0.00

Add funds **Auto-refill** **Funding methods**

Transaction history Category: Services

Amount	Description	Payment method	Payer	Date
\$ 2.50	Laundry - Washer	Debit card ending in 8210	Rohit Chaudhuri	2026-06-01
\$ 2.50	Laundry - Dryer	Debit card ending in 8210	Rohit Chaudhuri	2026-06-01
\$ 2345.67	Rent	Debit card ending in 8210	Rohit Chaudhuri	2026-05-31
\$ 2.50	Laundry - Washer	Debit card ending in 8210	Rohit Chaudhuri	2026-05-25

Add funds

Amount: \$ 20.00

Pay with: Debit card ending in 8210

Cancel **Confirm**

Funding methods

Saved Methods

Debit card ending in 8210

+ Add new card

Close

Testing Plan

Overview:

- Unit testing:
 - Test-driven development should be employed.
 - Unit test automation should be written alongside the code and run locally frequently.
- Integration testing:
 - Manual integration tests should be run after implementing every major feature.
 - Tests should be automated wherever possible.
- End-to-end testing:
 - Manual end-to-end tests should be run prior to every milestone submission, once the code is “frozen”, at least one day before the submission due date.
- Note: The test plans may change as we refine the design.

Testing tools:

- Frontend: Selenium, Mocha, Chai
- API: Postman
- Backend: Mocha, Chai

Plan for integrating testing frame-works:

-Selenium provides browser testing with JavaScript (and also with TypeScript) so we will mainly use it to test if the components are loaded on the webpage. It is possible to use React in tandem with Selenium, so we will test that the correct React components are loaded after an action with Selenium.

-Mocha is a testing framework for JavaScript, and Chai is an assertions library. We can use the two in tandem to create unit tests for the backend as well as the frontend as Mocha and Chai can both run in browser and in node.js.

-For API-testing we will mainly use Postman, e.x: testing that API calls return expected responses,

Frontend test plan:

The following specifications should be verified in the user interface (UI):

For this, we will mainly use Selenium, Chai, Mocha.

- All pages:
 - Test case 1: Style
 - 1) Setup: None
 - 2) Execution: Open any page of the web application.
 - 3) Validation: All UI elements are displayed with the correct style.
- All forms:
 - Test case 1: Form buttons
 - 1) Setup: None
 - 2) Execution: Open any form of the web application.
 - 3) Validation: There are two buttons for cancelling and confirming changes.
 - Test case 2: "Confirm" button function
 - 1) Setup: Access a form.
 - 2) Execution: Press the "confirm" button.
 - 3) Validation: Changing any value and pressing "confirm" exits the form and adds or changes the pertinent information correctly.
 - Test case 2: "Confirm" button function
 - 1) Setup: Access a form.

- 2) Execution: Press the “cancel” button.
 - 3) Validation: Changing any value and pressing “cancel” exits the form and does not change any information.
- Login page:
 - Test case 1: UI Elements
 - 1) Setup: None
 - 2) Execution: Open the login page.
 - 3) Validation: There are two text input fields present (e-mail and password) and one button input (log in)
 - Test case 2: Login function
 - 1) Setup: Open the login page.
 - 2) Execution: Enter a valid e-mail and password combination in their respective fields, then press the “Log in” button.
 - 3) Validation: Logging in as a valid resident/building manager user opens the resident/building manager dashboard page.
- Personal information form:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid user, then press the user icon.
 - 3) Validation:
 - There are two text input fields present (e-mail and password) and one button input (log in)
 - There are editable text or numerical input fields for name, apartment number, email and phone number.
- Payment information form:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid user, then navigate to the “Payments” page, then press “Payment Methods”.
 - 3) Validation: There are editable text, numerical or date input fields for card number, cardholder name, expiration date and card verification value.
- Shared facilities page, resident view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid resident, then navigate to the “Facilities” page.
 - 3) Validation:
 - There is a list of named items corresponding to each shared facility with an indication of current availability, and a button to book a time slot.
 - The “reserve” button is disabled if the shared facility is unavailable.
 - Test case 2: “Reserve” button

- 1) Setup: Log in as a valid resident, then navigate to the “Facilities” page.
 - 2) Execution: Press the “Reserve” button for an available facility.
 - 3) Validation: Pressing the “book a time slot” button opens a form with the fields “date” and “time”. Pressing “confirm” books a time slot for the chosen facility and the current user.
- Test case 3: Booked facility availability.
 - 1) Setup: Ensure there is at least one booked facility.
 - 2) Execution: Log in as a valid resident, then navigate to the “Facilities” page. Then, wait until the current booking for that facility ends.
 - 3) Validation: Booked facilities should automatically be displayed as unavailable during their booked time, and then displayed as available once that time lapses.
- Shared facilities page, building manager view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid building manager, then navigate to the “Facilities” page.
 - 3) Validation: There is a list of named items corresponding to each shared facility with an indication of current availability, and buttons to add, edit and remove facilities. Each item has a checkbox to select it.
 - Test case 2: “Add Device” form
 - 1) Setup: Log in as a valid building manager, then navigate to the “Facilities” page.
 - 2) Execution: Press the “Add Device” button
 - 3) Validation: The add button opens a form with the fields “name” (text input) and “type” (dropdown menu). Pressing “confirm changes” adds a new item to the list.
 - Test case 3: “Edit Device”
 - 1) Setup: Log in as a valid building manager, then navigate to the “Facilities” page.
 - 2) Execution: Press the “Edit” button in a listed device.
 - 3) Validation: The edit button opens a form with the fields “name” (text input) and “type” (dropdown menu) and an “available” checkbox. Pressing “confirm changes” with the “available” checkbox ticked lists the item as available, and without the checkbox ticked the item is listed as unavailable but is still present in the list. Pressing the “Remove” button removes the device from the list.
- Payment portal view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid resident, then navigate to the “Payments” page.

- 3) Validation: The payment portal lists all previous payments made for building facilities, with their date and amount. There is a “Make Payment” button that opens the payment form.
 - Test case 2: Payment form
 - 1) Setup: Log in as a valid resident, then navigate to the “Payments” page.
 - 2) Execution: Press “Make Payment”
 - 3) Validation: In the payment form, there are editable fields identical to those in the payment information form, in addition to an “amount” numerical field, and a “Make Payment” button that attempts to make a payment.
 - Test case 3: Accepted payment
 - 1) Setup: Log in as a valid resident, then navigate to the “Payments” page.
 - 2) Execution: Press “Make Payment”, fill in all mandatory fields with valid values, choose a valid card as the payment method, then press “Pay”.
 - 3) Validation: If the payment is accepted, then a confirmation page should appear indicating the payment information, amount, and a confirmation code.
 - Test case 4: Rejected payment
 - 1) Setup: Log in as a valid resident, then navigate to the “Payments” page.
 - 2) Execution: Press “Make Payment”, fill in all mandatory fields with valid values, choose a valid card that will reject the payment or an invalid card as the payment method, then press “Pay”.
 - 3) Validation: If the payment is rejected, then a page indicating that the payment was rejected and the reason for rejection should appear with a “dismiss” button, which brings the user back to the payment portal.
- Maintenance requests page, resident view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid resident, then navigate to the “Maintenance” page.
 - 3) The resident’s previous maintenance requests are listed, with a description, date and status (one of “New”, “Contractor Requested” and “Resolved”) for each item on the list. There is a “Create Request” button that opens the maintenance request creation form, and “Edit” and “Remove” buttons.
 - Test case 2: Maintenance request creation form
 - 1) Setup: Log in as a valid resident, then navigate to the “Maintenance” page.
 - 2) Execution: Press the “Create Request” button.
 - 3) Validation: The maintenance request creation form has the input fields “category” (dropdown menu, indicating the general type of issue e.g.

- “plumbing”, “HVAC”, etc.), “location” (text input), “description” (text input), and priority (dropdown menu, containing at least two values, one of which being “emergency”). Pressing “confirm” in the maintenance request creation form creates a new entry in the maintenance requests list with a “new” status.
- Test case 3: Maintenance request edition form
 - 1) Setup: Log in as a valid resident, then navigate to the “Maintenance” page. Ensure the user has created at least one maintenance request.
 - 2) Execution: Press the “Edit” button for one of the listed maintenance requests.
 - 3) Validation: The “edit” button opens a form identical to the maintenance request creation form, with the fields filled in with the existing values for that maintenance request. Pressing “confirm” in this form edits the information associated with that request without modifying its status.
 - Test case 4: “Remove” button
 - 1) Setup: Log in as a valid resident, then navigate to the “Maintenance” page. Ensure the user has created at least one maintenance request.
 - 2) Execution: Press the “Remove” button for one of the listed maintenance requests.
 - 3) Validation: The “remove” button erases the maintenance request.
 - Maintenance requests page, building manager view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid building manager, then navigate to the “Maintenance” page. Once the main “Maintenance Requests” page is validated, press the “Edit” button in an existing maintenance request.
 - 3) Validation: In the building manager view, all maintenance requests are listed with the same information as shown in the resident view, with the addition of the apartment number for each entry. All functions in all forms are identical in the building manager view, except: In the maintenance request edit form, there is an additional dropdown menu “status” that changes the status of the request being edited.
 - Notices page, resident view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid resident, then navigate to the “Notices” page.
 - 3) Validation: In the resident view, there is a list of all notices the user can view. Notices not yet opened by the user are marked as “not read”.
 - Notices page, building manager view:
 - Test case 1: UI elements
 - 1) Setup: None
 - 2) Execution: Log in as a valid building manager, then navigate to the “Notices” page.

- 3) Validation: In the building manager view, there is a list of all notices, with a checkbox for each item to select it, a “Create Notice” button, and “Edit” and “Remove” buttons for each notice.
 - Test case 2: Notice creation form
 - 1) Setup: Log in as a valid building manager, then navigate to the “Notices” page.
 - 2) Execution: Press the “Create Notice” button.
 - 3) Validation: Pressing the “create” button opens the notice creation form with the fields “title” (text input) and “body” (multi-line text input), and “confirm” and “cancel” buttons. Pressing “confirm” publishes the notice so that it is visible for all residents.
 - Test case 3: Notice edition form
 - 1) Setup: Log in as a valid building manager, then navigate to the “Notices” page. Ensure there is at least one notice present.
 - 2) Execution: Press the “Edit” button in one of the present notices.
 - 3) Validation: Pressing the “edit” button opens a form identical to the notice creation form, with the fields filled in with existing values for that notice. Pressing “confirm” in this form edits the information associated to that notice and enables the “not read” indications for all residents.
 - Test case 4: “Remove” button
 - 1) Setup: Log in as a valid building manager, then navigate to the “Notices” page. Ensure there is at least one notice present.
 - 2) Execution: Press the “Remove” button in one of the present notices.
 - 3) Validation: The “remove” button erases the notice.
- Residents page:
 - Test case 1: Access to the Residents page
 - 1) Setup: None
 - 2) Execution: Log in as a resident, try navigating to the “Residents” page. Log out and try the same procedure logging in as a building manager.
 - 3) Validation: It is only possible to view the Residents page when logged in as a building manager.
 - Test case 2: UI elements:
 - 1) Setup: None
 - 2) Execution: Log in as a building manager and navigate to the “Residents” page.
 - 3) Validation: A list of apartments and their residents is shown.

Backend and API test plan:

We will use Mocha, Chai to test the backend, and Postman to test the API calls. The following specifications should be verified either manually, using test automation, or simulating API calls with Postman:

Some definitions:

Records represent entries in databases which can store maintenance requests, notices, items in shared facilities, etc.

We will first create tests on the backend side using Mocha and Chai, before running end-to-end tests with Postman.

- **All forms and lists (Mocha, Chai, Postman):**
 - **Test Plan 1:** Adding a record to the database:
 - Description: When a request associated with a “create” function in a form is received, then all the received information is added to a new record in the pertinent database table. The new row must be complete and no information other than the received must be added.
 - 1) Setup: No setup required.
 - 2) Execution: Add a new record A to the database.
 - 3) Validation: Assert that A is in the database.
 - **Test Plan 2:** Removing record from database:
 - Description: When a request associated with an “remove” function in a form is received, then the record and all references to it are deleted from the pertinent database table or tables.
 - 1) Setup: Add record A to the database.
 - 2) Execution: Remove record A from the database
 - 3) Validation: Assert that A is not in the database.
 - **Test Plan 3:** Updating record from database:
 - Description: When a request associated with an “edit” function in a form is received, then all the received information is added to the correct existing record in the pertinent database table. The schema of the row or table must not change. No new rows must be created or deleted.
 - 1) Setup: Add record A to the database.
 - 2) Execution: Edit record A
 - 3) Validation: Assert that edits to A are persisted in the database.
 - **Test Plan 4:** Retrieving 1 record from database:
 - Description: When a range of items in a list is requested, the returned list is complete, missing no fields or items, and has no items other than the requested.
 - 1) Setup: Add records A, B, C, D to the database.
 - 2) Execution: Call method requesting for record A.
 - 3) Validation: Assert that record A is returned.
 - **Test Plan 5:** Retrieving no records from database:
 - Description: When a range of items in a list is requested, the returned list is complete, missing no fields or items, and has no items other than the requested.
 - 1) Setup: No setup required.
 - 2) Execution: Call method requesting for records A.

- 3) Validation: Assert that an empty list is returned.
 - **Test Plan 5:** Retrieving records from database:
 - Description: When a range of items in a list is requested, the returned list is complete, missing no fields or items, and has no items other than the requested.
 - 1) Setup: Add records A, B, C, D to the database.
 - 2) Execution: Call method requesting for records A, C
 - 3) Validation: Assert that records A and C are returned in a list.
- **Login and authentication (Mocha, Chai, Postman):**
 - **Test plan 1:** For every user, their password is stored encrypted.
 - 1) Setup: No setup required
 - 2) Execution: Add user account A and password a.
 - 3) Validation: Verify that A and encrypted(a) are stored in the database and not a.
 - **Test plan 2:** When a log in request is sent, the password is transmitted in an encrypted manner.
 - 1) Setup: No setup required.
 - 2) Execution: Call login() method with username A and password a as an argument.
 - 3) Validation: Assert that login() sends encrypted(a), an encrypted version of a to the backend.
 - **Test plan 3:** If an invalid login is entered, the log in request is rejected.
 - 1) Setup: Create combinations of invalid inputs: (valid user, invalid password), (invalid user, valid password), (invalid user, valid password)
 - 2) Execution: Call the login() method with each combination as arguments.
 - 3) Validation: Assert the login() method returns a “rejected” status
 - **Test plan 4:** Requests for personal or payment information are only accepted if the request comes from the authenticated user whose information is being requested.
 - 1) Setup: Add valid account A and corresponding encrypted password a to the database.
 - 2) Execution: Call getInformation() with A and a as arguments.
 - 3) Validation: Assert that getInformation returns an “accepted” status
- **Business logic consistency (Mocha, Chai):**
 - **Description:** There are never any entries in the database table storing shared facility bookings that have the same facility, date and time.
 - There are several cases:
 - Case 1: Entries A and B have same facility,
 - Case 2: Entries A and B have same data
 - Case 3: Entries A and B have same time
 - Case 4: Entries A and B share the same values for more than 1 entry.

- **Test Plan 1:** Add entry B with duplicate value in one or more of the fields: facility, date and time:
 - 1) Setup: For each case, create two entries A and B. Add entry A to the database.
 - 2) Execution: For each case, call addEntry() with B as an argument.
 - 3) Validation: Assert for each case that A is in the database, and B is not.
- **Test Plan 2:** Add entry B after removal of pre-existing entry A with duplicate value
 - 1) Setup: Create entries A and B such that they share one or more duplicate values in one or more of the fields: facility, date, and time. Add entry A to the database.
 - 2) Execution: Remove entry A. Then add entry B.
 - 3) Validation: Assert that A is not in the database, and B is.

Task Assignment

Name	Experience	Role
James	C# .NET EntityFramework backend, SQL databases, some React/MongoDb/Node, Old C++, Java, Postman, Wireshark, VisualStudio, ExpressJS, IntelliJ, Linux	Backend/Testing/Database config or Architecture
Janet	Python, JavaScript, Node.js, Java, SQL, TypeScript, Mocha, Chai, HTML, C	-Backend/Database/Testing -Frontend/Data Visualization/Testing -API Testing
Gale	JS/TS, C++, SQL, MongoDB, Java, Selenium, Postman, some HTML/CSS	-Middleware -Frontend/Testing
Zhuoyan	JS, C++, SQL, TS, Java, Flask, HTML, Node.js, REST APIs, Ubuntu	Middleware/Frontend/DB/(Help other members in Backend if I had some ideas)